

Excelente trabajo en la ronda 3: el fix del join `roles(\*)` quedó aplicado en el GET path, el `.gitignore` quedó ASCII limpio, el import duplicado se borró, hiciste todos los opcionales que sugerimos (README actualizado, `.env.example` raíz borrado, `run-auth.sh` con perfil supabase, `\@Email / \@Size` en `RegisterRequest`). El servicio ahora compila, bootea contra Supabase real y `\actuador/health` responde 200. **Los 3 blockers de ronda 3 están cerrados.**

PERO el smoke test contra Supabase real descubrió **2 blockers nuevos** que antes eran invisibles, más **2 issues importantes** que el próximo dev se va a comer.

Aspecto	Estado	
Boot contra Supabase real	OK (6.2s, perfil <code>supabase</code> activo)	<code>\actuador/health</code>
OK (200)	Build + tests	OK (16s)
Validaciones (<code>\@Email</code> , <code>\@Size</code> , <code>\@NotBlank</code>)	OK (400 cuando corresponde)	Wrong password
OK (401)	<code>\auth/register</code> happy path	ROTO (500 PGRST204)
<code>\auth/login</code> happy path	ROTO (500 42P01, tabla <code>refresh_tokens</code> no existe)	<code>\auth/refresh</code>
ROTO (500 mismo motivo)	<code>\auth/logout</code>	ROTO (500 mismo motivo)
JWT keys del <code>.env</code> se cargan	NO (cae al path efímero aunque estén en env)	<code>\health</code>
ROTO (403, no whitelisted en <code>SecurityConfig</code>)	<code>\auth/jwks</code> (sin <code>.well-known/</code>)	ROTO (403)

El `SupabaseRestRefreshTokenStoreAdapter` intenta hacer POST a `\refresh\_tokens` y la tabla **no existe en Supabase**.

Confirmado por introspección directa del schema (GET `\rest/v1/`):

Tablas presentes: `users`, `login`, `user\_has\_roles`, `roles`, `client`, `delivery`,
`delivery\_routes`, `categories`, `products`, `orders`,
`order\_items`, `restaurant`, `usuarios`
Tablas ausentes: `refresh\_tokens` ← acá muere el login

El stack trace al hacer login con creds válidas (`cliente@demo.cl / 123456`):

ERROR: 42P01 – relation \"public.refresh_tokens\" does not exist
at SupabaseRestRefreshTokenStoreAdapter.save(...)
at RefreshTokenManager.issueFor(...)
at LoginUserService.login(...)

Cómo arreglarlo

Tenés dos opciones. **Recomendada la A** (más simple, menos invasiva):

Opción A — agregar la tabla `refresh\_tokens` a Supabase (desde el SQL editor del panel o con `psql`):

```
create table if not exists public.refresh\_tokens (  
  id          bigserial primary key,  
  user\_id    bigint not null references public.users(id),  
  token\_hash text  not null unique,  
  expires\_at timestampz not null,  
  created\_at timestampz not null default now(),  
  revoked\_at timestampz  
);  
create index if not exists refresh\_tokens\_user\_idx on  
public.refresh\_tokens(user\_id);  
create index if not exists refresh\_tokens\_hash\_idx on  
public.refresh\_tokens(token\_hash);
```

Opción B — repurposing de `login`: si la tabla `login` ya tiene columnas de refresh, extendéla. Pero antes validá con el equipo líder que no haya otro servicio consumiéndola.

Cómo verificar

Después de aplicar el DDL:

```
\# Login con un seed user (cliente\@demo.cl \/ 123456)  
curl -s -X POST http://localhost:8081/auth/login \  
  -H "Content-Type: application/json" \  
  -d '{"login":"cliente\@demo.cl","password":"123456"}' \  
  -w "\nHTTP\=%{http\_code}\n\  
  
\# Esperado: 200 con {session\_token, access\_token, refresh\_token, userId}  
\# NO 500
```

Si devuelve 200 y los tokens, el bug está cerrado.

Tu fix de ronda 3 agregó el campo `@JsonProperty("user\_has\_roles")` al record `UserRow` para que el GET pudiera deserializar el join anidado. **El mismo record se usa en el POST**, y como `user\_hasRoles` es `null` en `save()`, Jackson lo serializa como `"user\_has\_roles":null` en el body. PostgREST lo rechaza:

```
ERROR: PGRST204 – Could not find the 'user\_has\_roles' column of 'users' in the schema  
cache  
at SupabaseRestUserRepositoryAdapter.save(SupabaseRestUserRepositoryAdapter.java:52)
```

Cómo arreglarlo

Opción A — mínimo (recomendada para esta ronda):

Agregá `\@JsonInclude(JsonInclude.Include.NON\_NULL)` al record `UserRow`. Eso evita que campos `null` se serialicen.

```
import com.fasterxml.jackson.annotation.JsonInclude;

@JsonInclude(JsonInclude.Include.NON\_NULL)
public record UserRow(
    Long id,
    String email,
    String rut,
    String name,
    String lastName,
    String phone,
    String photo,
    Instant createdAt,
    \@JsonProperty(\"user\_has\_roles\") List<UserHasRoleNestedRow> userHasRoles
) {}
```

Opción B — más limpia (recomendada para el futuro):

Partí los DTOs en:

- `UserCreateRow` (solo campos escribibles, sin `user\_has\_roles`)
- `UserReadRow` (con `user\_has\_roles` para hidratar roles)

Después cambiá `save()` para usar `UserCreateRow` y los métodos de lectura para `UserReadRow`.

Cómo verificar

```
\# First register (email válido) – esperado 201 (o 200)
TIMESTAMP=\$(date +%s)
EMAIL=\"verify-r4-\\${TIMESTAMP}@test.com\"
curl -s -X POST http://localhost:8081/auth/register \\\
  -H \"Content-Type: application/json\" \\\
  -d \"{\\\"email\\\":\\\"\\${EMAIL}\\\",\\\"password\\\":\\\"Segura1234\\\",\\\"name\\\":\\\"Verify\\\",\\\"lastName\\\":\\\"R4\\\",\\\"phone\\\":\\\"+56912345678\\\"}\" \\\
  -w \"\\nHTTP\\=%{http\_code}\\n\"

\# Duplicate – esperado 4xx (409 o 400 con error de duplicado)
curl -s -X POST http://localhost:8081/auth/register \\\
  -H \"Content-Type: application/json\" \\\
  -d \"{\\\"email\\\":\\\"\\${EMAIL}\\\",\\\"password\\\":\\\"Segura1234\\\",\\\"name\\\":\\\"Verify\\\",\\\"lastName\\\":\\\"R4\\\",\\\"phone\\\":\\\"+56912345678\\\"}\" \\\
  -w \"\\nHTTP\\=%{http\_code}\\n\"
```

Si el primer devuelve 2xx y el segundo 4xx (no 500), el fix funciona.

El servicio tiene `JWT\_PRIVATE\_KEY` y `JWT\_PUBLIC\_KEY` en `auth-service/.env` con PEMs RSA reales. Pero `RsaKeyProvider` los ignora y genera claves efímeras al boot, descartando las del env.

Evidencia (verificada):

- El endpoint `/auth/.well-known/jwks.json` devuelve un JWKS

- El modulus del JWKS **no coincide** con la clave pública del `.env`
- Resultado: el servicio **no puede validar tokens emitidos por otro sistema**. Cualquier integración queda rota.

Causa probable: PEM multi-línea vía env-var en Spring Boot. El wrapper de Gradle o Spring Boot trunca los `\n` y solo lee la primera línea del PEM.

Cómo arreglarlo

Opción A — reemplazar `\n` por `\\n` literal en el `.env`:

```
JWT\_PRIVATE\_KEY=\"-----BEGIN PRIVATE KEY-----\nMIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCwggSjAgEAAoIBAQ...\n-----END PRIVATE KEY-----\"
JWT\_PUBLIC\_KEY=\"-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA...\n-----END PUBLIC KEY-----\"
```

Y en el código, reemplazar los `\n` literales antes de pasárselos al `KeyFactory`:

```
String privateKeyPem = privateKeyPemRaw.replace("\\n", "\n");
```

Opción B — leer los PEMs desde archivos en vez de env-var (más limpio):

```
JWT\_PRIVATE\_KEY\_PATH=/run/secrets/jwt-private.pem
JWT\_PUBLIC\_KEY\_PATH=/run/secrets/jwt-public.pem
```

Y en el adapter:

```
String pem = Files.readString(Path.of(path));
```

Cómo verificar

Después del fix:

```
## Capturar JWKS actual
curl -s http://localhost:8081/auth/.well-known/jwks.json > /tmp/opencode/jwks.json

## Extraer modulus y comparar con la clave pública del .env
openssl rsa -in /path/to/public\_key.pem -pubout -modulus -noout 2>&1
```

Si los moduli coinciden, el fix funciona.

El `HealthController` existe pero `SecurityConfig` solo whitelist `/actuator/health/*\*`. El endpoint custom `/health` está muerto en prod.

Cómo arreglarlo

Opción A — agregar `/health` al `permitAll()` de `SecurityConfig`:

```
http.authorizeHttpRequests(auth \-> auth
    .requestMatchers("/actuator/health/*\"", "/health\"", "/auth/.well-known/
jwks.json").permitAll()
    // ... resto
);
```

Opción B — borrar el `HealthController` si no lo usás (el `/actuator/health` ya cubre).

Paso 1 — Crear tabla `refresh_tokens` en Supabase (PRIORIDAD MÁXIMA)

Conectate al panel de Supabase (o `psql` con la URL de Kong) y aplicá el DDL del Bloqueante 1. Sin esto, login/refresh/logout siguen rotos. Confirmá con un `select count(*) from refresh_tokens;`.

Paso 2 — Fix PGRST204 en `UserRow`

Editá `auth-service/src/main/java/com/flashdrop/auth/infrastructure/adapters/outbound/persistence/supabase/dto/UserRow.java`

Opción A del Bloqueante 2 (agregar `@JsonInclude(NON_NULL)`).

```
./gradlew :auth-service:clean build \-Dorg.gradle.java.home=/home/pelle/jdk/jdk-21.0.2
\--no-daemon
\# BUILD SUCCESSFUL
git add auth-service/src/main/java/com/flashdrop/auth/infrastructure/adapters/outbound/
persistence/supabase/dto/UserRow.java
git commit \-m "\"fix(auth): prevent null user_has_roles from leaking into POST body
(PGRST204)\""
```

Paso 3 — Whitelist `/health` en `SecurityConfig`

Editá el archivo de `SecurityConfig` y agregá `/health` al `permitAll()`. Commit:

```
git commit \-m "\"fix(auth): whitelist /health endpoint in SecurityConfig\""
```

Paso 4 — Fix JWT keys del env (opcional pero recomendado)

Aplicá la Opción A del Importante 1 (literal en el `.env` + replace en código). Si no lo hacés en esta ronda, marcalo explícito como **TODO** porque bloquea cualquier integración multi-servicio.

Paso 5 — Push + comentario en el PR

```
git push origin feat/supabase-rest-migration
\# Comentario en PR \#2: "\"Round-4 fix: refresh_tokens table created in Supabase,
UserRow JsonInclude fixed, /health whitelisted. Login/register/refresh/logout now
return 2xx.\""
```

(Esta sección se mantiene del feedback de ronda 3 — es la receta que ya te pasamos.)

Antes de probar los fixes, asegurate de tener el ambiente configurado:

1. Crear el `.env` desde la plantilla

```
cd juniors/junior-1-auth
cp auth-service/.env.example auth-service/.env
```

Editá `auth-service/.env` y verificá que tenga estas 3 líneas (el resto puede quedar con los defaults):

```
SUPABASE_URL=http://supabasekong-wymwq8rktid7ov678oe4va90.76.13.169.150.sslip.io
SUPABASE_SERVICE_ROLE_KEY=<pedirle al líder del equipo – NUNCA versionar>
SPRING_PROFILES_ACTIVE=supabase
```

Lo que NO debe quedar: `DB_URL=jdbc:postgresql://...`, `DB_USER`, `DB_PASSWORD`. Si las ves, borralas.

2. Setear `JAVA_HOME`

```
java -version
\# Esperado: openjdk version "21.x.x"
```

Si no tenés JDK 21 en PATH:

```
export JAVA_HOME=/path/a/tu/jdk-21
export PATH="$JAVA_HOME/bin:$PATH"
```

3. Arrancar el servicio

```
./gradlew :auth-service:bootRun -Dspring.profiles.active=supabase
```

Cuando veas `Started AuthServiceApplication in N seconds`:

```
curl http://localhost:8081/actuator/health
\# Esperado: {"status":"UP","groups":[...]}
```

-
- **Lo que hiciste bien en ronda 3:** 3 blockers + 5 opcionales en un solo commit, todos aplicados. Compila, bootea, conecta a Supabase real.
 - **Lo nuevo que aparece:** el smoke test contra Supabase real descubrió que login/refresh/logout dependen de una tabla que no existe, y que register se rompió por el fix de ronda 3. 4 issues, 2 críticos.
 - **Próxima ronda esperada:** después de aplicar el DDL de `refresh_tokens` + el fix de `UserRow`, todos los happy paths deberían dar 2xx. Si pasa, evaluación final.

Cuando termines, subí los cambios a la misma rama y avisame para volver a probar.

- **Spring Boot** `\@JsonInclude` docs: <https://www.baeldung.com/jackson-deserialize-json-unknown-properties>
- **Multi-line env-var en Spring**: <https://stackoverflow.com/questions/38754237/how-to-inject-a-string-with-newlines-in-spring-using-application-yml>
- **PostgREST embedded resources**: https://docs.postgrest.org/en/v12/references/api/resource_embedding.html
- **Patrón Supabase REST en catalog-service** (referencia del repo del junior-3): <https://github.com/javier-sudo/catalog-service-java/tree/main/src/main/java/com/flashdrop/catalog/infrastructure/adapters/outbound/persistence/supabase>